```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv('tecdiv.csv')
```

```
df.head()
df.info()
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 64 entries, 0 to 63
Data columns (total 11 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   Timestamp             64 non-null     object
 1   Email Address         64 non-null     object
 2   Name                  64 non-null     object
 3   Email                 64 non-null     object
 4   Roll no               64 non-null     object
 5   PRN No.               64 non-null     object
 6   Mobile No.            64 non-null     int64
 7   First year:   Sem 1   64 non-null     float64
 8   First year:   Sem 2   64 non-null     float64
 9   Second year:   Sem 1  64 non-null     float64
 10  Second year:   Sem 2  64 non-null     float64
dtypes: float64(4), int64(1), object(6)
memory usage: 5.6+ KB
```

	Mobile No.	First year: Sem 1	First year: Sem 2	Second year: Sem 1	Second year: Sem 2
count	6.400000e+01	64.000000	64.000000	64.000000	64.000000
mean	8.623097e+09	8.834219	9.095469	9.292031	9.377187
std	9.132070e+08	11.187839	11.171986	0.528523	0.495185
min	7.028870e+09	0.000000	0.000000	6.900000	7.200000
25%	7.766559e+09	7.237500	7.655000	9.050000	9.140000
50%	8.805720e+09	8.260000	8.400000	9.445000	9.450000
75%	9.335094e+09	8.802500	9.115000	9.645000	9.725000
max	9.975810e+09	95.000000	95.000000	9.910000	9.950000

STEP 1: Missing Values & Logical Cleaning

```
df.isnull().sum()#check for missing value
```

	0
Timestamp	0
Email Address	0
Name	0
Email	0
Roll no	0
PRN No.	0
Mobile No.	0
First year: Sem 1	0
First year: Sem 2	0
Second year: Sem 1	0
Second year: Sem 2	0

dtype: int64

```
df.drop(['Timestamp', 'Email', ], axis=1, inplace=True)
```

```
df = df.drop(index=[42,43,48,52,62,35])
```

Feature Engineering

```
df['Average_Score'] = df[['First year:   Sem 1','First year:   Sem 2','Second year:   Sem 1','Second year:   Sem 2']].mean(axis
```

```
df.head()
```

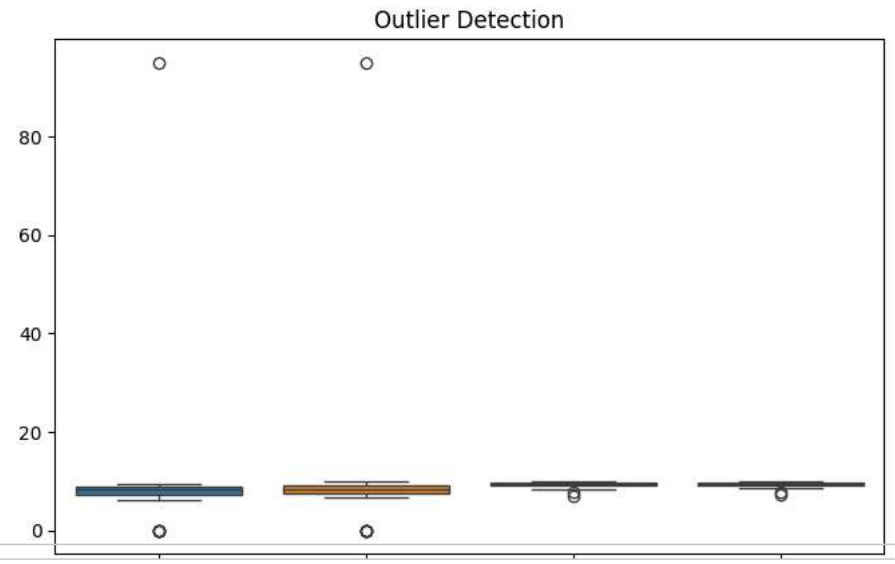
Show hidden output

```
def performance(x):
    if x >= 9:
        return 'Excellent'
    elif x >= 8:
        return 'Good'
    elif x >= 7:
        return 'Average'
    else:
        return 'Poor'

df['Performance'] = df['Average_Score'].apply(performance)
```

Outlier Detection

```
plt.figure(figsize=(8,5))
sns.boxplot(data=df[['First year:   Sem 1','First year:   Sem 2','Second year:   Sem 1','Second year:   Sem 2']])
plt.title("Outlier Detection")
plt.show()
```




```
def remove_outliers(df, cols):
    for col in cols:
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1

        lower = Q1 - 1.5 * IQR
        upper = Q3 + 1.5 * IQR

        df = df[(df[col] >= lower) & (df[col] <= upper)]
    return df

df = remove_outliers(df, ['First year:   Sem 1','First year:   Sem 2','Second year:   Sem 1','Second year:   Sem 2'])
```

Data Transformation

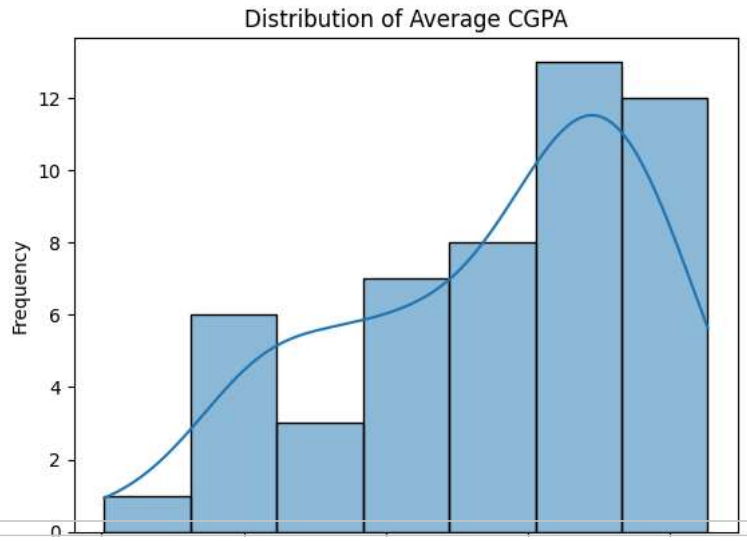
```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

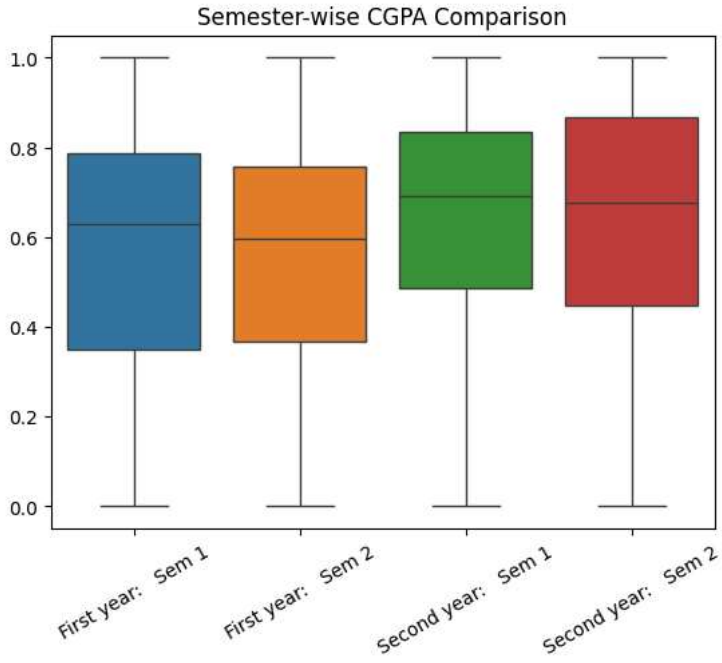
df[['First year:   Sem 1','First year:   Sem 2','Second year:   Sem 1','Second year:   Sem 2']] = scaler.fit_transform(
    df[['First year:   Sem 1','First year:   Sem 2','Second year:   Sem 1','Second year:   Sem 2']]
)
```

Visualization

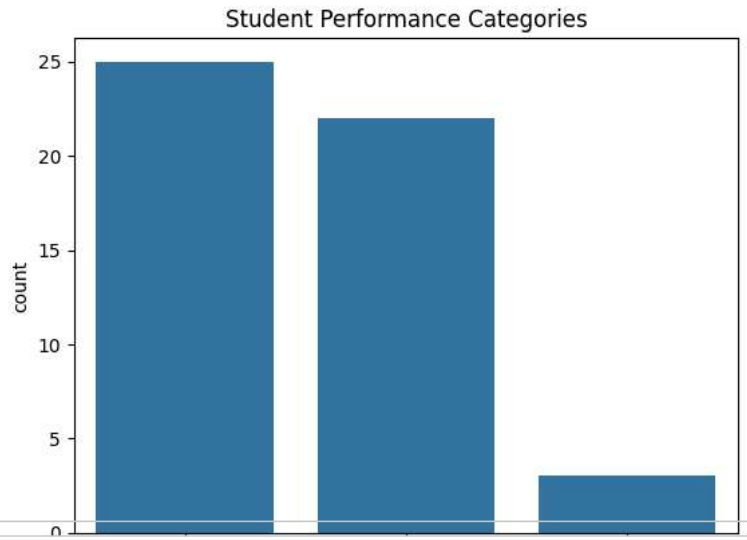
```
import seaborn as sns
sns.histplot(df['Average_Score'], kde=True)#CGPA Distribution (Histogram)
plt.title("Distribution of Average CGPA")
plt.xlabel("CGPA")
plt.ylabel("Frequency")
plt.show()
```



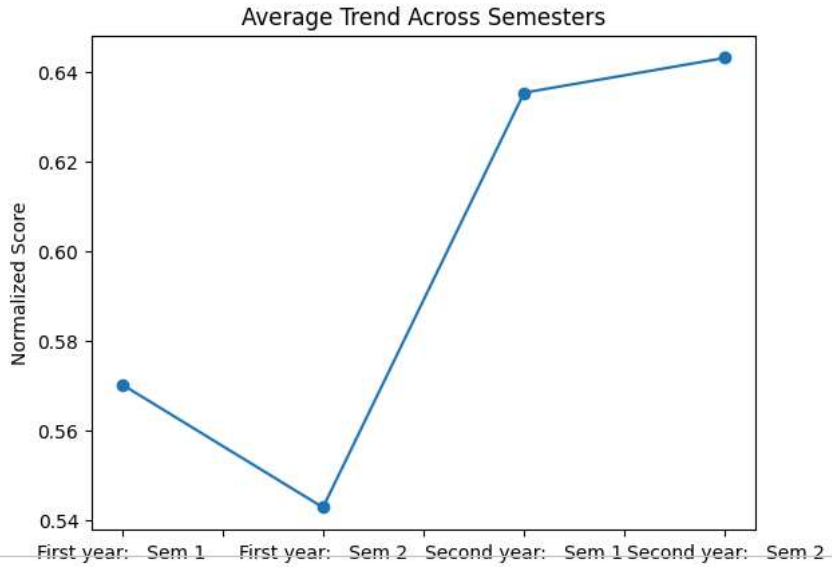

```
sns.boxplot(data=df[['First year:   Sem 1',
                     'First year:   Sem 2',
                     'Second year:   Sem 1',
                     'Second year:   Sem 2']])
plt.title("Semester-wise CGPA Comparison")
plt.xticks(rotation=30)
plt.show()
```



```
sns.countplot(x='Performance', data=df)
plt.title("Student Performance Categories")
plt.show()
```




```
df[['First year:   Sem 1','First year:   Sem 2','Second year:   Sem 1','Second year:   Sem 2']].mean().plot(marker='o')
plt.title("Average Trend Across Semesters")
plt.ylabel("Normalized Score")
plt.show()
```




```
df.to_csv("cleaned_tecdiv.csv", index=False)
```